

DMN4DQ + : Optimising data repair to enhance data usability

Álvaro Valencia-Parra ^{a,*}, Ángel Jesús Varela-Vaca ^a, Luisa Parody ^b,
Ismael Caballero ^c, María Teresa Gómez-López ^a

^a Dpto. Lenguajes y Sistemas Informáticos, IDEA Research group, Universidad de Sevilla, Sevilla, Spain

^b Dpto. Métodos Cuantitativos, Universidad Loyola Andalucía, Sevilla, Spain

^c Information Systems and Technologies Institute, University of Castilla, La Mancha, Ciudad Real, Spain

ARTICLE INFO

Keywords:

Data quality
Data repair
Corrective actions
Constraint programming

ABSTRACT

Data quality has become crucial in decision-making and data analysis. There is an intrinsic relationship between data quality and usability; however, acceptable levels of data quality depend on the contextual requirements and operational priorities of an organisation. The context of use, business needs, and the organisation's appetite for risk all influence the usability of data. Achieving adequate levels of usability sometimes requires specific corrections, which can be costly and may incur have far-reaching consequences. This paper introduces the concept of target usability, by representing the minimum level of usability determined by business analysts at which data records can be used without compromising organisational performance. When data records fail to meet this threshold, a combination of corrective actions can be implemented to improve both quality and usability. To achieve near-optimal outcomes, the data quality analyst can effectively combine these sets of actions. This paper proposes DMN4DQ +, an extension of DMN4DQ, where the optimal combination of corrective actions can be derived from the application of constraint optimisation techniques based on the data quality rules described in decision models, the cost model of the actions, and on the usability profile of the record to be improved. The development of a technological stack has conveniently supported DMN4DQ +, and it has been evaluated using a real dataset, thereby demonstrating its applicability and performance.

1. Introduction

Data quality is more important than ever for organisations because data has become the cornerstone asset for strategic, tactical, and operational decisions (Pérez-Álvarez et al., 2018; Shankaranarayanan & Cai, 2006). Therefore, organisations must consider only data with suitable quality levels to prevent, or at least alleviate, undesirable consequences (Gualo et al., 2021; Subramanian, 2025; Wang et al., 2024). Examples of these consequences include incorrect decision-making, operational performance issues, damage to corporate image, and increased undesirable costs. Failure to address these issues can result in significant resource losses. Redman reported that the cost per item of poor quality data for US companies reaches approximately 3.1 USD trillion per year (Redman, 2016a).

Even when there is a close relationship between data quality and “data usability”, both concepts must be understood at different levels. Data usability refers to the degree to which data can be potentially used

in a specific domain without risking the performance of the organisational business processes, according to the appetite for risk. In contrast “data quality” is typically defined as the degree to which data is fit for intended use (Shankaranarayanan & Cai, 2006; Valencia-Parra et al., 2019). In this respect, data with an inadequate level of quality may be entirely usable (e.g., there is no other data available about a customer to deliver their products and meet their needs, and somebody must provide the product nonetheless). On the other hand, data with an adequate level of quality may be unusable due to an unacceptable trade-off between cost and benefits. However, it can be stated that the higher the levels of data quality, the more usable the data becomes. Consequently, there is a need to continuously assess and improve data quality to ensure it meets the minimum required level of usability for its intended purpose.

However, it is necessary to identify both those data records with inadequate levels of quality (those records are considered *non-conformance* data) and their underlying root cause. Identifying non-conformance data

* Corresponding author.

E-mail addresses: avalencia@us.es (Á. Valencia-Parra), ajvarela@us.es (Á.J. Varela-Vaca), mlparody@uloyola.es (L. Parody), Ismael.Caballero@uclm.es (I. Caballero), maytegonmez@us.es (M.T. Gómez-López).

<https://doi.org/10.1016/j.eswa.2025.129170>

Received 30 October 2024; Received in revised form 23 July 2025; Accepted 24 July 2025

Available online 30 July 2025

0957-4174/© 2025 The Author(s). Published by Elsevier Ltd. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

requires measuring and assessing the levels of quality against a set of business rules that describe the adequate use of the data for the organisation's intended purpose.

Depending on the nature of the underlying root cause (i.e., violation of specific business rules) of the non-conformance data, data quality analysts must identify, select, and apply the most suitable set of *corrective actions* for data improvement. Among these actions, the most widely used is data cleaning (Ilyas & Chu, 2019; Subramanian, 2025), which aims to detect and correct data values that violate any of the stated business rules. Examples of data cleaning actions include editing misspellings and filling in missing values. It is worth highlighting that the execution of corrective actions can affect one or more records and that their performance depends on the order of execution and always implies a cost (Redman, 2016b) (e.g., in terms of human and computational resources). Identifying the optimal combination or set of corrective actions involves analysing not only the dependencies between the stated business rules related to data quality but also the wide range of possible corrective actions to apply, the cost of implementing these actions, and the nature of the data repository. It is a significant challenge to provide data analysts with appropriate tools to assess design alternatives and choose specific corrective actions that optimise (i.e., reduce) the total cost of improving the data. Due to the high number of possibilities and aspects that must be considered, we propose using the Constraint Programming paradigm for both modelling and solving this optimisation problem (Rossi et al., 2006a).

To address this research challenge, we extended the DMN4DQ framework (Valencia-Parra et al., 2021) to identify the optimal set of corrective actions that maximise the usability of data records in a given dataset while balancing cost-effectiveness. The DMN4DQ framework, grounded in the *Decision Model and Notation* (DMN) standard, enables data quality analysts to model business rules for data quality measurement and assessment in a systematic manner. Decision Model and Notation is a standard designed by the Object Management Group (OMG, 2019) to model and execute decision logics. It has been designed to be easily understandable by both business and technical users. It provides a visual notation for decision requirements and a formal language for decision logic. These features make DMN suitable for applications in data quality management, where the specification, traceability, and automation of complex business rules are critical.

The primary contribution of this paper is DMN4DQ+, an extension of the DMN4DQ framework. DMN4DQ+ is methodological work that supports data quality analysts to evaluate data quality and identify optimal corrective actions to meet user requirements, thereby leveraging the concept of a *usability profile*, which characterises data usability in specific contexts based on business rules. It models corrective actions and their costs and facilitates targeted data usability improvements, supported by a technological stack validated with real datasets. Using Constraint Programming (CP) as an Artificial Intelligence technique, DMN4DQ+ identifies optimal actions transparently and traceably, which aligns with business needs. Unlike opaque machine learning models, CP ensures decisions are auditable and based on explicit rules.

To clarify our proposal, a simple case study from Valencia-Parra et al. (2021) is first given, while Section 5 evaluates its applicability in a more complex scenario. The dataset, a CSV catalogue of Amazon Web Services server instances,¹ includes 21 attributes. For data quality assessment, there are three key attributes are: (1) InstanceFamily, a string categorising the instance (e.g., Memory optimised, General purpose); (2) ClockSpeed, a string indicating CPU speed (e.g., "3.0 GHz"); and (3) Memory, a string for RAM size ("16 GiB").

The rest of the paper is structured as follows: Section 2 outlines the foundations of DMN4DQ+, including DMN4DQ. Section 3 introduces new DMN4DQ+ elements. Section 4 details the use of Constraint Programming to identify optimal actions to improve data usability.

Section 5 explains the real-case application of DMN4DQ+ using the technical stack developed herein. Section 6 reviews relevant prior work and is followed by the conclusions and future work.

2. Background

To introduce our approach, a description of the case study, which will be discussed throughout the article, is provided. The most relevant concepts regarding DMN4DQ, data quality improvement, and Constraint Programming are also explained.

2.1. Data improvement: data cleansing

The concept of "data improvement" goes beyond the idea of "data cleansing". "Data cleansing" aims to modify the value of specific attributes in the data to match the expected value. However, this is not the only means for organisations to improve their data. For example, in another context, a redesign of the business process that utilises data may be required, together with the training of individuals who play the role of data consumers to prevent them from making mistakes. All these actions can be considered improvements of data and go beyond the concept of mere "data cleansing".

According to ISO 8000-61 (2022), data quality improvement involves the process of identifying data quality issues based on the results of data quality assessment to correct *non-conformance* data. However, for the sake of understandability, in the data management domain, the expressions *data error* or even *data quality problem* (Gualo et al., 2021; Ilyas & Chu, 2019) are typically preferred over the more accurate term of *non-conformity*, which is widely used in quality management. The frequency and periodicity of the appearance of *data errors* or *non-conformance data* make it possible to identify two types of patterns: random data errors and systematic (Bertossi, 2011; Ilyas & Chu, 2019). Systematic data errors are typically caused by failures in the design of information systems or in the business process in which the data is used. Random data errors are typically caused by random factors (e.g., a specific issue with a data collector device or due to inevitable human error). Fixing random data errors requires data cleaning techniques (e.g., HoloClean Boeckling et al., 2022, Renuver Breve et al., 2022, and Derand Song et al., 2020). On the other hand, the correction of systematic data errors can need one type of corrective action to fix the "systematic" cause (e.g., redesigning the business processes or training data quality analysts) and a second type of corrective action (e.g., data cleansing) to fix the errors that the "systematic" cause has produced over time. In this paper, our focus is solely on data cleansing techniques.

Ilyas and Chu (2019) identified two main activities as part of a data cleansing project: (i) **Non-conformity detection**, aimed at identifying those non-conformance data records (i.e., those that violated the stated business rules); and (ii) **Non-conformity repairing or correction**, which consists of transforming or changing the values of the data record so that they meet the stated business rules. To render the identification of non-conformance records and their corresponding root cause more accessible, DMN4DQ was introduced; a summary will be presented in Section 2.2. It is necessary to select (in the case of multiple alternatives) and apply a set of potential corrective actions to the records that need to be updated to clean the data. However, data cleansing comes at a cost, and selecting the optimal combination of feasible and cost-effective corrective actions remains an open problem that requires further investigation. To produce an approach to the optimal set of corrective actions, the foundations of the Constraint Optimisation paradigm are recalled in Section 2.3.

2.2. DMN4DQ in a nutshell

DMN4DQ was first presented in Valencia-Parra et al. (2021) as a DMN-based framework for generating recommendations on the usability of each data record in a dataset. Decision Model and Notation (DMN)

¹ <https://www.kaggle.com/datasets/akashsarda/aws-ec2-pricing-data>

is a modelling language and standard notation established by the Object Management Group (OMG) to delineate decision rules (OMG, 2019). It enables the modelling of decisions, which can be customised to meet the specific requirements. To this end, DMN simplifies the declarative description and formalisation of decisions using the “if-then” structure, where if the input satisfies a constraint, then an output is obtained, through the use of tables to represent each decision rule, where each row is an ‘if-then’ structure. To date, DMN has been used in various contexts, but it is also applicable to the assessment of data quality. Furthermore, DMN enables for combining various decision tables to be combined hierarchically, where the output of one decision can serve as the input for another. This fact enables decision-making rules to become more complex. Using the combination of DMN tables, the meaning of usability can be tailored to meet user requirements. Usability is based on the level of data quality according to the business needs and the organisation’s appetite for risk. The generation of these recommendations involves the definition of several types of *Business Rules for Data Decisions* (BR.DD) structured according to the business perception of data quality for the intended use of the data. One of the conceptual pillars of DMN4DQ involves the differentiation between the concepts of *measurement* and *assessment*. During the *measurement* process, the goal is to determine the compliance of the data records against stated data

requirements (expressed as “*Business Rules for Data Values*” (BR.DV)). Business Rules for Data Values tend to analyse the data format, for instance, for the case study presented above and as exemplified in Fig. 1, the outputs BR.DV.01 and BR.DV.02 are “false” if ClockSpeed and Memory are “null”; the output of BR.DV.03 and BR.DV.04 are “false” if the format of ClockSpeed and Memory fail to follow the specific patterns; and the output of BR.DV.05 is an Integer according to the values of Memory, ClockSpeed, and InstanceFamily.

The results of the evaluation of these BD.DVs are used as the input for the “*Business Rules for Data Quality Measurements*” (BR.DQM). For example, the output of the evaluation of BR.DV.01 and BR.DV.02 is used as the input to evaluate BR.DQM.Completeness to measure the completeness of the data. On the other hand, the outputs of the evaluation of BR.DV.03 and BR.DV.04 are used as the input to evaluate BR.DQM.Accuracy to measure the accuracy in obtaining the values 0, 50, or 100.

The output produced by the evaluation of these BD.DQMs is used as the input for the assessment. During the *assessment*, the measurements obtained for every data quality characteristic are aggregated according to the data quality requirements (expressed as “*Business Rules for Data Quality Assessment*” (BR.DQA)) to determine the overall level of quality of a given record. For example, the output of the evaluation of

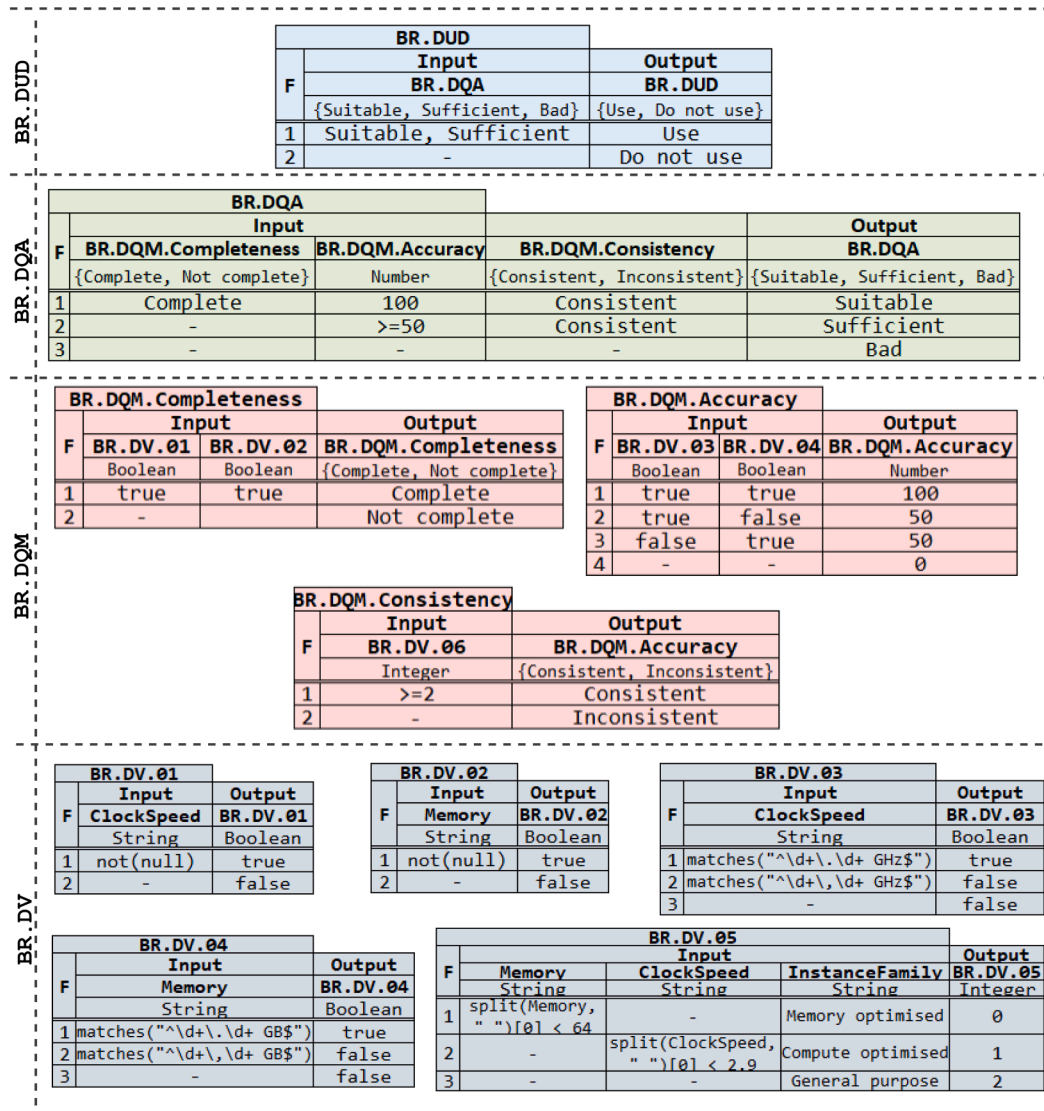


Fig. 1. DMN Hierarchy Diagram.

BR.DQM.Completeness, BR.DQM.Accuracy, and BR.DQM.Consistency is utilised to ascertain whether the assessment is suitable, sufficient, or poor. Based on the results of the assessment and taking into account both the business necessities and the organisation's appetite for risk, expressed through "Business Rules for Data Usability Decisions" (BR.DUD), a recommendation is generated regarding the usability of the data.

2.3. Constraint optimisation problems

To determine the most suitable combination of actions to transform an unusable record into a usable record, the application of the Constraint Programming (CP) paradigm (Rossi et al., 2006b) is proposed. A discipline within Artificial Intelligence (AI), CP is particularly adept at addressing problems characterised by a need to satisfy a set of constraints. While not all problems in Computer Science are faced with CP, many specific challenges within various subfields can be effectively modelled and solved using CP techniques. This fact includes, but is not limited to, problems in optimisation, scheduling, resource allocation, and decision-making under constraints. In these contexts, CP offers a framework for the identification of solutions that adhere to a given set of rules or limitations. A Constraint Optimisation Problem (COP) is a Constraint Satisfaction Problem (CSP) where an optimisation function determines the best solution from the set of possibilities according to the function to be optimised. The basis of CP stands on the resolution of a CSP. Therefore, to better understand what constitutes a COP is, a definition is introduced.

Definition 1. A CSP represents a reasoning framework consisting of variables, domains and constraints $\langle V, D, C \rangle$, where V is a set of n variables $\{v_1, v_2, \dots, v_n\}$ whose values are taken from finite domains $\{D_{v_1}, D_{v_2}, \dots, D_{v_n}\}$ respectively, and C is a set of constraints on their values. The constraint $c_k(x_{k_1}, \dots, x_{k_m})$ is a predicate defined on the Cartesian product $D_{k_1} \times \dots \times D_{k_m}$. This predicate is true iff the assignment of the value of each variable x_{k_i} satisfies the constraint c_k .

The search for solutions for a CSP is based on the concept of instantiation. An assignment of a variable, or instantiation, is a *variable-value* pair (x, a) which represents the assignment of the value a to the variable x . An instantiation of a set of variables is a tuple of ordered pairs, where each sorted pair (x, a) assigns the value a to the variable x . A tuple $\langle (x_1, a_1), \dots, (x_i, a_i) \rangle$ is consistent if it satisfies all the constraints formed by the variables of the tuple. Several possible solutions can be found to satisfy a CSP, but sometimes only the most suitable solution must be obtained: that which optimises (maximises or minimises) the solution. In these cases, a COP must be considered.

Definition 2. A COP is a CSP in which the solutions found optimise (minimise or maximise) an objective function f .

The typical Knapsack problem (Kellerer et al., 2004) can be described as a COP as shown in Fig. 2. The knapsack problem is formed of a set

of constants given as inputs of the problem (list of weights and costs of the potential elements to be included), variables to represent whether the element is to be included or not, the constraints on the limit of the possible values that the variables can take, and the objective to maximise, the total cost in this case. Every possible combination of elements that can be included in the knapsack is analysed, and those that maximise a *totalCost* without exceeding a *maximumWeight* are selected. The use of COP enables the problem to be modelled instead of having to define an algorithm, since the COP solver will analyse the possible values of the variables through the application branch and bound techniques.

The problem addressed in this work has a well-defined combinatorial structure with clearly specified constraints, making it naturally suited to be modeled using CP. Additionally, CP's applicability spans logistics, scheduling, planning, networks, cybersecurity and other domains (Geibinger et al., 2024; Green et al., 2024; Jiménez-Piqueras et al., 2023; Laghrissi et al., 2016; Urli & Kilby, 2017; Varela-Vaca et al., 2021), thereby demonstrating its effectiveness despite computational challenges.

3. DMN4DQ+, a solution for efficient data repairing

Before entering into an in-depth description of DMN4DQ+, let us recall that data usability largely depends on data quality; consequently, to increase the usability of data, it is possible to improve the levels of data quality. However, data quality improvement does indeed come with a cost, which can be optimised in a cost-driven way. This cost optimisation can be achieved by selecting only those corrective actions which minimise the total cost of the improvement projects. To the best of our knowledge, no investigations have been conducted to address the relationship between data usability, data management, and cost-driven data quality improvement, which renders our investigation highly innovative. After having participated in several academic and practitioner data quality management projects, our motivation focused on providing data quality analysts with the necessary methodological and technological tools to support cost-driven data quality improvement, thereby increasing the usability of the data at a controlled cost. After having developed DMN4DQ and testing it in real-world contexts, it was decided to extend this framework to support cost-driven data quality improvement. To provide a unified outcome, it was decided to build DMN4DQ+ on top of DMN4DQ, and, consequently, DMN4DQ+ supersedes DMN4DQ for the data quality improvement projects. Hence, DMN4DQ+ aims to find the optimal cost-benefit set of corrective actions to convert a set of records having the same "usability profile" to achieve a "usability target" of "usable" value in accordance with the stated business rules. To achieve these data improvements using data repair, this paper extends DMN4DQ by adding the required components to address data quality improvement management concerns, including the notion of cost. In this section, the new add-ins for DMN4DQ+ are presented and formalised.

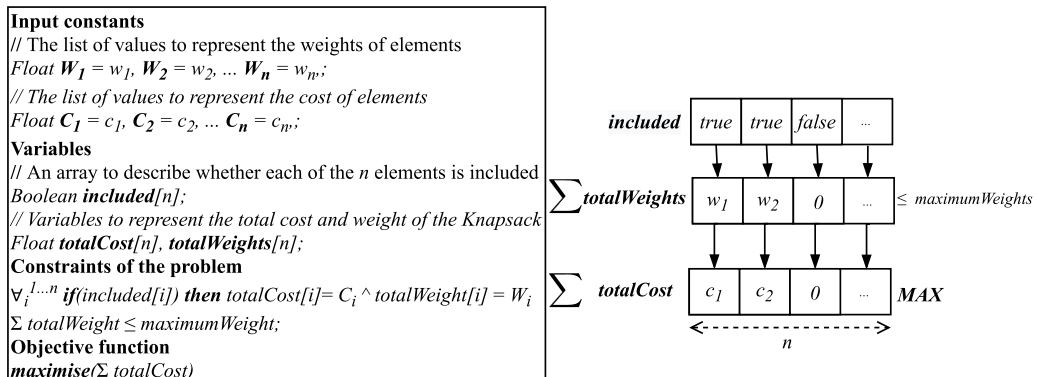


Fig. 2. Schema of the Knapsack COP model.

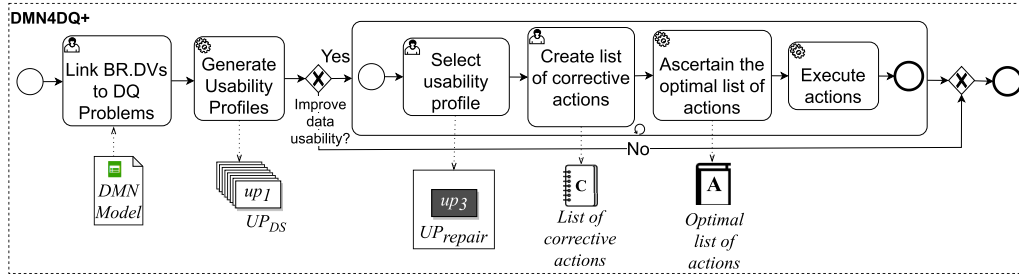


Fig. 3. DMN4DQ+ method.

3.1. DMN4DQ+ method

To facilitate the understanding of the proposal, the method is described using the workflow shown in Fig. 3.

Six elements are involved in the workflow are:

- **Dataset:** This contains the records whose usability is to be determined and improved. An example of such a dataset is presented for Amazon Web Services in Section 2.2.
- **DMN model:** This contains the hierarchies of business rules developed following the DMN4DQ methodology (Valencia-Parra et al., 2021). An example of a DMN hierarchical model is presented in Fig. 1.
- **List of corrective actions:** These are the actions that can change the dataset quality. Corrective actions that can be applied to the example include string replacement and data reformatting.
- **Usability profile (UP) of each record of the dataset:** This is the description of the conditions under which a record should be used for a given purpose. An example of a usability profile is BR.DUD = 'Do not use', BR.DQA = 'Bad', BR.DQM.Completeness = 'Not complete', BR.DQM.Accuracy = 0, BR.DQM.Consistency = 'Inconsistent', BR.DV.01 = false, BR.DV.02 = true, BR.DV.03 = false, BR.DV.04 = true, BR.DV.05 = 1, and BR.DV.06 = 'Inconsistent'. The formal definition is presented below.
- **Target usability:** This describes the minimum amount of usability (to be defined by business analysts) at which the data record should be used in the organisation. An example can be BR.DUD = 'Use', BR.DQA = 'Sufficient', BR.DQM.Completeness = 'Complete', BR.DQM.Accuracy = 50, BR.DQM.Consistency = 'Consistent', BR.DV.01 = true, BR.DV.02 = true, BR.DV.03 = false, BR.DV.04 = true, BR.DV.05 = 2, and BR.DV.06 = 'Consistent'.
- **Optimal list of actions:** This is a subset of the list of corrective actions to perform to achieve the target usability with the minimum cost. Examples of corrective actions include modifying the data format and managing the relationship with data providers to prevent null data values. The final list of actions will be obtained using Constraint Programming.

In summary, there are five steps that the DMN4DQ+ method introduces (which are described in the following subsection) are:

1. **Link the business rules for data values (BR.DVs) to identify data quality problems.** The possible data quality problems are related to the values of the tuples, and hence are also associated with the BR.DVs. Since each corrective action is intended to address one or more data quality problems (rooted in the violation of BR.DV(s)), it is possible to track those BR.DV(s) that could be affected by a specific corrective action.
2. **Generate for every record the corresponding “usability profiles”** (see Section 3.3) The results of the usability recommendations generated using DMN4DQ (Valencia-Parra et al., 2019; Valencia-Parra et al., 2021) are taken as the basis for these usability profiles.

3. **Select the “usability profiles” whose records must be repaired.** The records with the same “usability profile” are grouped to later apply the same corrective actions.
4. **Create a list of corrective actions.** These actions are intended to address specific data quality issues that may be detected in data attributes. The actions will be applied to the records that have the selected “usability profiles”.
5. **Determine the optimal combination of the feasible and cost-effective set of corrective actions** to improve the levels of data quality towards the desired level of usability of the record selected by the user. From among the existing approaches towards finding the optimal combination, our proposal utilises the Constraint Programming paradigm.

3.2. Link the business rules for data values to identify data quality problems

Data quality problems include anomalies in datasets that can trigger the non-usability of the data according to the measurement and the assessment described in each case (Quindroit et al., 2023; Woodall et al., 2014). Typically, it is easier for stakeholders to identify common data quality problems (e.g., missing values, incorrect values, incorrect syntax, and domain violations). However, to address the data quality problems, data quality analysts should focus on both identifying the business rules for data values (BR.DVs) and determining those that have been violated, which cause the data quality issues. To a certain extent, it can be stated that data quality problems should be characterised as a problem of non-conformity for one or more data quality characteristics or dimensions (de Almeida et al., 2013). For the identified data quality characteristics, it is, therefore, possible to identify and group the corresponding business rules for the data values (Caballero et al., 2022). Consequently, business stakeholders and data quality analysts can work collaboratively to represent data quality problems as violations of business rules for data values (BR.DVs) for the attributes involved (see Fig. 4). Business stakeholders must specify which data quality problems are the most damaging to the business; data quality analysts must then determine which business rule violations need to be prioritised for repair.

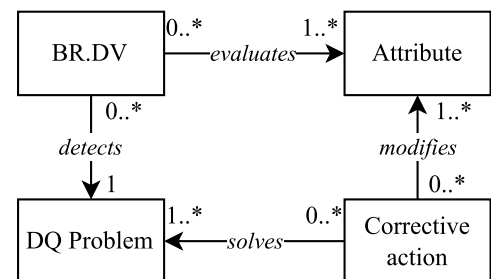


Fig. 4. Relationship between BR.DVs, data quality problems, corrective actions, and dataset attributes.

Table 1
Data quality problems associated with each BR.DV of the guiding example.

Implied Attributes	DQ Problem	Type of DQ Problem	DQ Dimension	BR.DV
ClockSpeed	Missing Value	SAST	Completeness	BR.DV.01
Memory	Missing Value	SAST	Completeness	BR.DV.02
ClockSpeed	Syntax Violation	SAST	Accuracy	BR.DV.03
Memory	Syntax Violation	SAST	Accuracy	BR.DV.04
Memory, ClockSpeed	Violation of functional dependency	MAST	Consistency	BR.DV.05

DMN4DQ can support this mapping between data quality problems and violations of business rules for data values.

Data quality problems can be classified according to their different levels of granularity (Oliveira et al., 2005; Quindroit et al., 2023; Woodall et al., 2014), depending on the number of attributes and tuples implied in such problems: (i) single attributes in single tuples (SAST); (ii) multiple attributes in single tuples (MAST); (iii) single attributes in multiple tuples (SAMT); and (iv) multiple attributes in multiple tuples (MAMT).

Table 1 illustrates these types of problems. For the SAST problem, it is assumed that a business rule, coded as BR.DV.01, exists for the *ClockSpeed* attribute, which states that a value must be present. Should this business rule be violated, then a problem arises, as is the case with a SAST issue. Similarly, it is assumed that the business rule coded as BR.DV.05 represents a functional dependency between the attributes *Memory* and *ClockSpeed*. If this business rule is violated for a single tuple, then there is a MAST problem. As an example of a SAMT problem, the case of the attribute *Memory* is assumed, for which the selected data type is *character* instead of *integer* resulting in the capture of invalid values for memory. Finally, as an example of MAMT, let us consider the business rules that have a functional dependency between the attributes *Memory* and *ClockSpeed*. The inadequate selection of datatype for *Memory* leads to a consistency problem for *ClockSpeed* in those tuples where *Memory* takes a value larger than ‘9’, which results in no valid value for *ClockSpeed*.

Given that the DMN4DQ is focused only on single tuples (also called records) due to the inherent limitations of DMN to represent Business Rules for Data Values, the types of problems that are detected by DMN4DQ and consequently dealt with by DMN4DQ+ are only SAST and MAST. This fact is, therefore, recognised as an inherent limitation of the DMN4DQ and DMN4DQ+ frameworks. However, this tuple can be extracted directly from the dataset, or generated using, for example, join or aggregation operators, thereby allowing the inclusion of multiple attributes of various tuples in a single tuple whose usability can be analysed.

For the guiding example presented in Fig. 1, Table 1 presents several data quality problems associated with each BR.DV.

3.3. Generate “Usability Profiles”

To introduce the concept of “usability profiles”, the guiding example is continued where the following data record is considered:

$r_1 = \langle \text{ClockSpeed} : \text{null}, \text{Memory} : 16,0GB, \text{InstanceFamily} : \text{“Memory optimised”} \rangle$.

To generate the “usability recommendation” for this record r_1 , the evaluation of the associated business rules gathered in the corresponding DMN tables is simulated (see Fig. 1):

1. The evaluation of the various BR.DVs generate the following results:
 - The evaluation of BR.DV.01 generates *false* due to *ClockSpeed* : *null*.
 - The evaluation of BR.DV.02 generates *true* since it is not null.
 - The evaluation of BR.DV.03 generates *false* since *ClockSpeed* is *null*.
 - The evaluation of BR.DV.04 generates *false* since *Memory* does not match the pattern indicated in row number 2 (note that the decimal point is given as “,” instead of “.”).

- The evaluation of BR.DV.05 is 0 due to *InstanceFamily*: “*Memory optimised*” and *Memory* not fulfilling the condition (note that, since it fails to follow the expected pattern, it is not possible to evaluate whether it fulfils the numerical condition).
2. The evaluation of the BR.DQMs generates the following results:
 - BR.DQM.Completeness evaluation generates *Not complete* since the evaluation of BR.DV.01 produced *false*.
 - BR.DQM.Accuracy evaluation generates 0 since the evaluation of both BR.DV.03 and BR.DV.04 generated *false*.
 - BR.DQM.Consistency evaluation generates *Inconsistent* since the evaluation of BR.DV.05 produced 0.
 3. The evaluation of the BR.DQAs generates the following results:
 - The evaluation of BR.DQA produces *Poor* since the evaluation of the BR.DQM.Completeness produced *Not complete*, the evaluation of BR.DQM.Accuracy generated 0, and the evaluation of BR.DQM.Consistency led to *Inconsistent*.
 4. Lastly, the evaluation of the BR.DUDs generates the following results:
 - The evaluation of the BR.DUD generates *Do not use* because the evaluation of BR.DQA generated *Not suitable* or *Sufficient*.

Visually, the backtrack of the generation of the usability of the recommendation of this record r_1 can be represented as shown in Fig. 5.

Usability profiles provide valuable information for the solving of data quality problems since they enable a better analysis of the problem’s root causes. Given that a record has a specific usability recommendation of “*Do not use*”, it is possible to identify, through a backwards analysis, **what** (aka the violation of the business rules for data values) provoked the problems.

This idea of retaining information regarding the traceability of the generation of the usability recommendation for a record enables the definition of the concept of **usability profile of a record**.

Definition 3 (Usability profile of a data record). Given a DMN model, D , the *usability profile* (up_{r_i}) for a data record r_i is a tuple $\langle \text{decision}, \text{assessment}, \text{measurements}, \text{observations} \rangle$ where:

- *decision* $\in D.BR.DUD.output$;
- *assessment* $\in D.BR.DQA.output$;
- *measurements* = $\{BR.DQM.Dim1: m_1, BR.DQM.Dim2: m_2, \dots, BR.DQM.Dimn: m_n\}$, where $BR.DQM.Dimi$ is the name of the table for the quality dimension, such that $m_i \in D.BR.DQM.Dimi.output$; and
- *observations* = $\{BR.DV.V1: v_1, BR.DV.V2: v_2, \dots, BR.DV.Vm: v_m\}$, where $BR.DV.Vi$ is the name of BR.DV table, such that $v_i \in D.BR.DV.Vi.output$.

Let us denote UP as the set of all possible “usability profiles” up_i that have been detected in DS after the generation of data usability recommendations. Various records may have the same “usability profile”. This fact enables the definition of **cluster**, leading to better management of data quality actions, as shown later in this paper.

Definition 4 (Cluster of Records with the same usability profile). Given a *usability profile* up_i , c_k is a cluster of records of the dataset $\{r_1 \dots r_j\}$, if $up(r_i) == up(r_j)$, and there does not exist a $r_k \notin \{r_1 \dots r_j\}$ where $up(r_k) == up(r_j)$.

Since every cluster $c_k \subset DS$ has the same usability profile up_i then the records r_i in c_k experience the same data quality problems. Consequently, records with the same usability profile can be grouped into

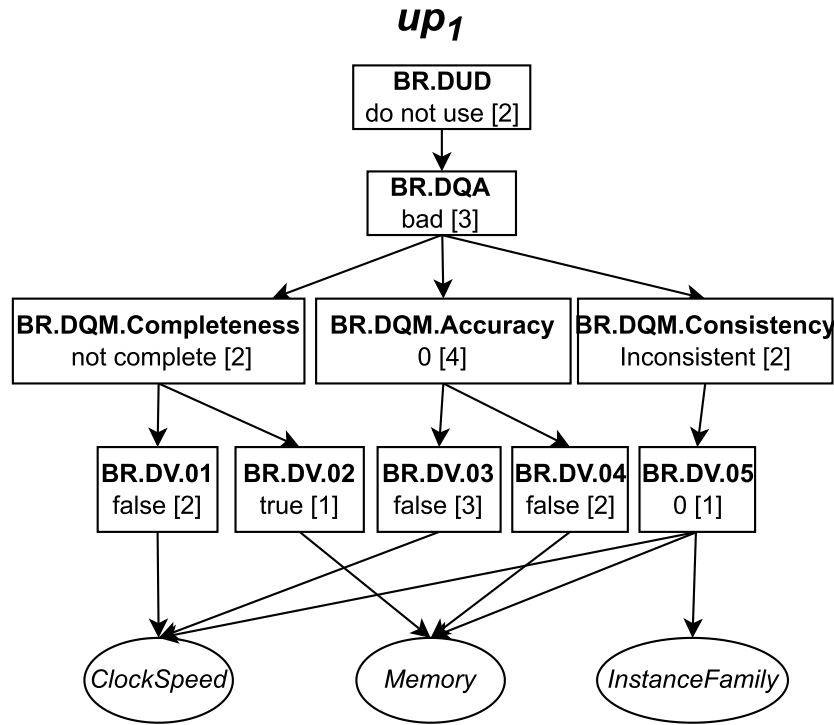


Fig. 5. Usability profile up_1 shows the value returned for each DMN table at each hierarchical level for the record r_1 . The row of the DMN table that was triggered is shown in brackets.

Clusters of Records to enable batch processing towards data quality improvement.

3.4. Create a list of corrective actions

The corrective actions can affect the set of attributes of the “cluster of records with the same usability profile”. When a “set of corrective actions” are applied, data usability can be improved to achieve the “usability target”. These terms are defined below.

Definition 5 (Usability Target). Let $tud \in D.BR.DUD$ be the usability target as the desired output of the evaluation of the usability recommendation given by a DMN model, D .

Let us assume that a cluster $c_k \subset DS$, with a usability profile up_1 , has a recommendation of “Do not use” subsequent to the evaluation of the corresponding $BR.DUD$. Let us assume that business stakeholders are interested in improving the usability from “Do not use” to “Use” for the c_k , given the usability target “Use”. Through the inspection of the “usability profile” of c_k , the objective therefore is to find the optimal

set of corrective actions that, once performed, ensure that all records r_i within c_k achieve the stated usability target “Use”.

To this end, the concept of “corrective action” is introduced in Definition 6.

Definition 6 (Corrective action for a usability profile). A corrective action ca for a given “usability profile” is a sequence of actions on the values of one or more attributes of the records r_i of a given cluster $c_k \subset DS$ to produce changes to achieve the stated usability target.

A corrective action is composed of several parts (see Table 3): (i) corrective action **name**, according to the last column of Table 2; (ii) the **data quality problem** to be repaired, and the associated **data quality dimension**, according to the second and third columns of Table 2; (iii) the list of **attribute(s)** to be repaired; (iv) the **operations** to be performed to achieve the usability target; (v) the **consequences** of applying the operation; (vi) the **requirements and conditions** that must be met for the operation to be performed; and (vii) the estimated **cost** of performing the operation.

Table 3 illustrates the various parts of the corrective actions and their structure for the guiding example.

Table 2
Set of Data Quality Problems and potential corrective actions.

Type	DQ Problem	DQ Dimensions	Proposed corrective actions
SAST	Missing value	Completeness	Data enrichment, data rules
	Syntax violation	Accuracy, Consistency	Regex string replacement, data reformatting
	Incorrect value	Accuracy	Data standardisation, data type conversion
	Domain violation	Completeness, Accuracy	Data rules, data standardisation
	Invalid substring	Accuracy	Data standardisation, regex string replacement
	Misspelling	Accuracy	Data standardisation, regex string replacement
	Imprecise value	Accuracy	Data standardisation, regex string replacement
MAST	Incorrect value	Accuracy	Data rules
	Inconsistent data format	Accuracy	Data rules, regex string replacement, data reformatting
	Semi-empty tuple (missing data)	Completeness	Data rules, data enrichment
	Violation of functional dependency	Accuracy, Consistency	Data rules
	Violation of business domain constraint	Accuracy, Consistency	Data rules

Table 3

A corrective action from the guiding example.

Part	Name	Description	Example
(i) Name	name	Name of the corrective action	Data rules for InstanceFamily attribute
(ii) DQ problem and dimension	dqProblem	DQ Problem to solve	Violation of functional dependency
	dqDimension	DQ Dimension implied	Consistency
(iii) Attribute(s)	attributes	Attribute to repair	[InstanceFamily]
(iv) Operation	operation	Operation to perform	Assign a value to the attribute InstanceFamily depending on the values of Memory and ClockSpeed
(v) Consequences	consequences	Consequences of applying the operation in the BR.DV in which attribute is implied.	{brdv: 04, rowTriggered: 3}
(vi) Requirements and conditions	requiredAttributes	List of attributes that are required to perform the operation.	[ClockSpeed, Memory]
	conditions	List of conditions that must be met before performing the operation.	BR.DV.01 must be true (row 1) AND BR.DV.02 must be true (row 1) AND BR.DV.03 must be true (row 1) AND BR.DV.04 must be true (row 1)
(vii) Cost	cost	Cost of performing the operation.	1

At this point, a limitation for the corrective actions needs to be assumed. Given the current restriction on DMN4DQ regarding the generation of recommendations for single records, the execution of corrective actions on affected attributes can only address SAST and MAST problems (see the first two columns of Table 2) to achieve a given usability target in DMN4DQ+. Before delving further into this example, a description of each of the corrective action parts is provided.

- **Name, DQ problem and dimension, attribute(s), and operation**

In accordance with Definition 6 and Table 3, the first part of a corrective action includes the name of the corrective action to be applied. It must consist of one of the corrective actions defined in Table 2, and it must be aligned to the following parts of the corrective actions: the DQ problem to be fixed (*dqProblem*) and the affected dimension (*dqDimension*). The attributes to be repaired must subsequently be specified (*attributes*). These attributes are then modified by the operation (*operation*) defined in the corrective action. The values of the attributes of the data records in the *Cluster of Records with the same usability profile* are modified according to the specified operation. Regarding the corrective action depicted as an example in Table 3, the attribute to be repaired is *InstanceFamily*. The data quality problem to be solved involves a functional dependency that has been violated. This problem affects the *consistency* data quality dimension. Essentially, the value of *InstanceFamily* depends on the value of the attributes *ClockSpeed* and *Memory*. Therefore, the corrective action is *data rules* (see Table 2), and the operation is intended to assign a value to *InstanceFamily*. These data rules take the following form: “If *Memory* $\geq 64\text{ GHz}$ then *InstanceFamily* = *Memory optimised*, else if *ClockSpeed* $\geq 2.9\text{ GHz}$ then *InstanceFamily* = *Compute optimised*, else *InstanceFamily* = *General purpose*”.

- **Consequences**

The consequence of the corrective action is related to the compliance of the BR.DV is associated with the data quality problem to which the corrective action refers. As can be observed in Table 3, the consequences have the following structure $\langle br.dv, rowTriggered \rangle$, where *br.dv* is the reference number of the BR.DV to which this consequence refers and *rowTriggered* is the numeric id of the row that is triggered once the value of the attribute has been repaired. Note that the user can specify more than one consequence if several BR.DVs are involved. Note also that, in this proposal, the user is responsible

for determining the complete list of implications for the corrective action.

Regarding the corrective action depicted in the example of Table 3, the consequences of repairing the *InstanceFamily* attribute, as determined by the operation to be performed, are that the evaluation of the DMN table BR.DV.04 would trigger the row with ID “3”, thereby returning the value “2”.

- **Requirements and conditions**

Specific corrective actions (e.g., data rules) may involve an operation that depends on the value of other attributes in the data record as input to repair the value of the data attribute. These dependencies can be modelled in DMN4DQ+ through *requiredAttributes* and *conditions*. The *requiredAttributes* enables the specification of the data attributes that are required to perform the corrective action, while the *conditions* allows the specification of which BR.DVs must be met for *requiredAttributes* to be employed. In other words, *conditions* enables the modelling of the degree to which data quality problems associated with *requiredAttributes* must be solved.

In order to identify the most suitable corresponding actions to be carried out over the affected attributes for known data quality problems (see Fig. 4), the identified set of data quality problems described by Woodall et al. (2014) and Oliveira et al. (2005), the corrective actions proposed by Woodall et al. (2014), and the data quality dimension classification performed by de Almeida et al. (2013) are employed. To better guide data quality analysts when proposing potential corrective actions, the third column in Table 2 is introduced as a map between SAST and MAST problems and corrective actions. Following the example in Table 3, the corrective action to be applied (that is, the data rules) requires the attributes *ClockSpeed* and *Memory*. To use these attributes, they must be complete and accurate. The *conditions* enable these necessities to be indicated for the corrective action to be applied. In this case, the evaluation of BR.DV.01 and BR.DV.02 must return true (i.e., *ClockSpeed* and *Memory* must not be null), and the evaluation of BR.DV.03 and BR.DV.04 must return true (i.e., *ClockSpeed* and *Memory* must fulfil the established format so that these attributes can be processed by this corrective action).

- **Cost of corrective action**

Lastly, the business expert must estimate and establish a cost for the execution of every type of corrective action over the dataset. This must be represented as a numerical value that indicates the cost in

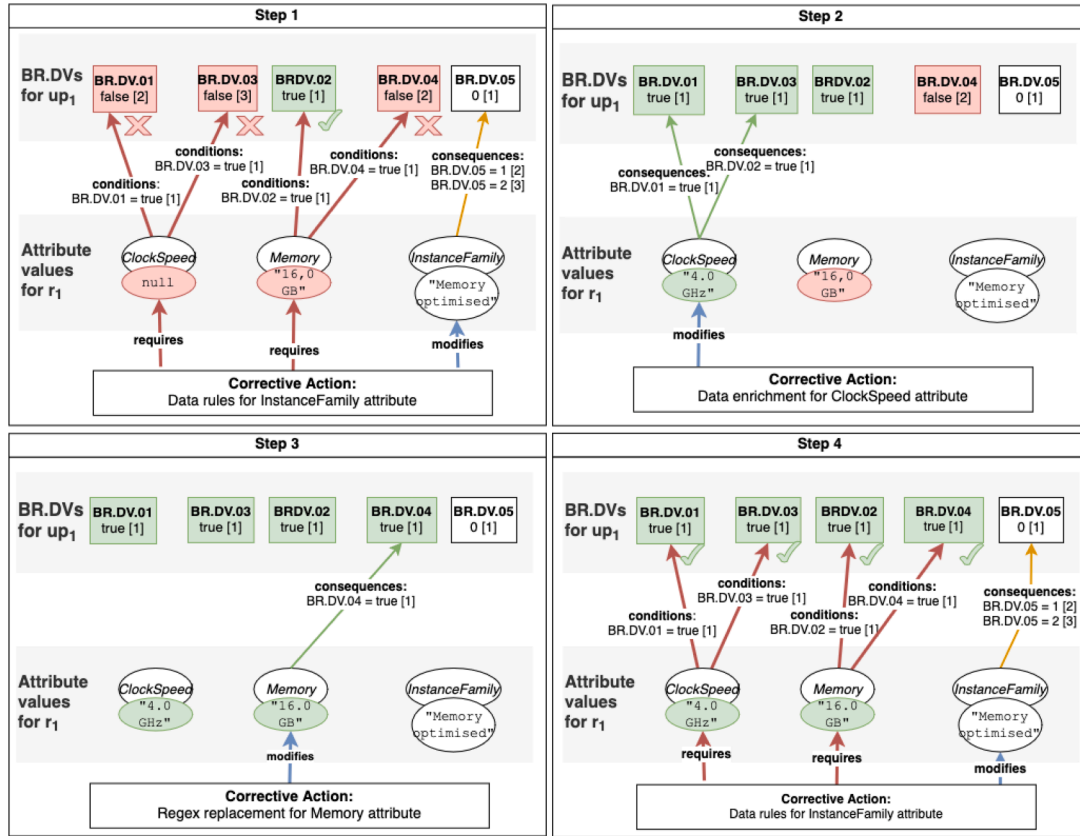


Fig. 6. Example of dependencies between corrective actions.

terms of both computational and/or human resources required to execute the action. Since estimating an exact cost can be complicated and overwhelming at this point, a qualitative estimation of the potential costs is proposed, which considers the experience of the data quality analysts. For instance, this description can be carried out using a Likert scale adjusted to the specific context of use to establish relative and subjective costs through the comparison of the difficulty implied in executing each rule. Therefore, in the remainder of the paper, Likert scales are employed, although using real cost estimation would not vary the sense of DMN4DQ+.

In the example of Table 3, the cost has been assigned a value of 1 on a scale of {1, 2, 3, 4, 5}, which implies the minimum cost. Compared to the performance of other corrective actions, this means that this corrective action does not entail a high human cost and has constant computational complexity.

3.5. Create a list of corrective actions

It is worth recalling that, when it comes to achieving the given usability target and depending on the nature of the root causes (business rules for data value violation), different combinations of corrective actions could be outlined following the various paths of the “usability profile”. These combinations are bounded by the conditions set in each corrective action. They enable DMN4DQ+ to include all the required actions whose consequences satisfy the conditions of other actions. Fig. 6 shows an example of how the conditions and requirements of one corrective action can be satisfied by applying other corrective actions.

- **Step 1.** In this step, the corrective action “Data rules for InstanceFamily attribute”, depicted in Table 3 is applied to r_1 , which forms part of the *Cluster of Records with the same usability profile* up_1 . First,

the conditions of the corrective action are evaluated. In this case, three conditions are not met. Therefore, this corrective action cannot be applied.

- **Step 2 and Step 3.** If the selected usability profile (up_1 in this example) does not meet the required conditions, then the optimal set of corrective actions whose consequences satisfy the required conditions is selected. In this case, two corrective actions could work: “Data enrichment for ClockSpeed attribute” and “Regex replacement for Memory attribute”. It should be borne in mind that the data steward must previously define all these corrective actions.
- **Step 4.** Lastly, the corrective action “Data rules for InstanceFamily attribute” can be applied since all the conditions are met.

This example reveals the strong connection between corrective actions and also illustrates the multiple alternative combinations of actions. The combinations strongly depend on the Clusters of Records with the same usability profile to be repaired and on the target.

Before delving into how to select the combination of corrective actions to apply, the concept of *list of corrective actions* (Definition 7) is introduced.

Definition 7 (List of corrective actions). A list of corrective actions $CA = \{ca_1, ca_2, \dots, ca_n\}$ groups the corrective actions that can be eligible to repair the data quality problems related to a Cluster of Records with the same usability profile.

The data steward is responsible for defining all these corrective actions. This list is crucial since how a record with a usability profile can be repaired to achieve a usability target depends on the set of corrective actions that can be applied to such records.

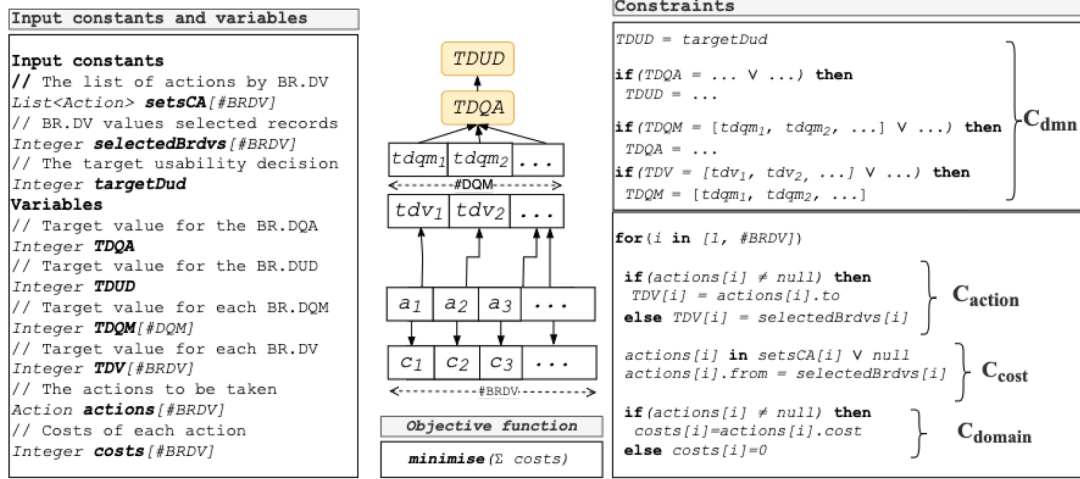


Fig. 7. Schema of the COP, constraints, variables and function to optimise.

4. Ascertaining the optimal combination of corrective actions the improvement of data usability

Our goal is to identify the cost-effective combination of corrective actions that enhances the usability of the dataset. This section presents a COP-based approach to determine the best corrective actions, by modelling the impact of each action's cost on the usability target in a declarative way. This COP examines possible action combinations for business data quality rules. The total cost is the sum of the costs of the selected actions: our intention involves minimising this sum. The model is an adaptation of the Knapsack problem (Section 2.3), where feasible actions are constrained by the rules of the DMN hierarchical table, rather than by a weight limit. Fig. 7 illustrates the input constants, variables, constraints, and objective function of the COP.

4.1. Constants

The COP relies on a series of input constants that are defined depending on the problem. The business expert must define the following elements: (i) the DMN data quality model; (ii) the usability profile to be repaired; (iii) the usability target; and (iv) the list of possible corrective actions. These elements are subsequently mapped to the COP model. The following elements must be included in a COP: the DMN data quality model, the usability profile to be repaired, the usability target, and the list of corrective actions. To facilitate understanding of the COP, certain constants are derived from the problem description, as described below.

- **#ACTIONS** is an Integer parameter that indicates the number of possible corrective actions to be applied as defined by the business expert.
- **#BRDV** is the number of DMN tables in the BR.DV hierarchical level.
- **lActions** is a representation of the list of corrective actions defined. Each corrective action is described as indicated in Table 3 and is mapped into the data structure *Action*, (Fig. 8), which is composed of the following attributes:

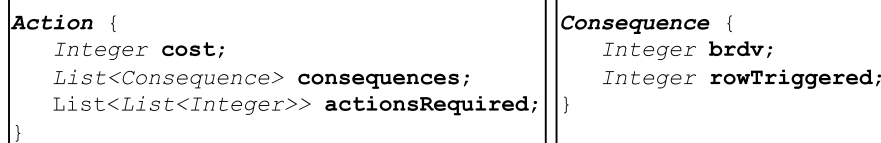


Fig. 8. Action and Consequence data structures.

- **cost** is directly mapped to the cost of the corrective action.
- **consequences** is a list composed of objects of type *Consequence*. Each object thereof is assigned to a consequence of the corrective action. A *Consequence* contains two attributes: *brdv* and *rowTriggered*. *brdv* describes the BRDV where the actions are applied, and *rowTriggered* describes the row of the DMN table affected by the actions. Information in greater detail is given in Section 3.4.
- **actionsRequired** is a list of lists of Integers. Each nested list indicates one possible combination of corrective actions that must be performed before the current corrective action is applied. The Integer represents the position of the action in lActions. This list is created before the creation of the COP, based on expert knowledge and the usability profile to be repaired. For example, to use the corrective action on InstanceFamily and obtain the value '2' in the evaluation of BR.DV.05, it is also necessary to apply a corrective action to ClockSpeed and Memory that modifies the output obtained for the evaluation of BR.DV.01 and BR.DV.02.
- **selectedBrdvs** is a list of Integer values. Each element of the list represents a BR.DV, and each value of the list is the row ID triggered for the usability profile that has been selected for repair.
- **targetDud** is an Integer value representing the row ID of the output value of the DMN table BR.DUD that corresponds to the user-selected target usability decision.

4.2. Variables

The variables have these purposes: (i) select the corrective actions to be applied; (ii) keep track of the costs of applying each corrective action; and (iii) keep track of the output values of the DMN values according to the selected corrective actions.

- **actions** is an array where each position *i* points to the *i*th corrective action from the list of corrective actions defined by the user, and indicates through a Boolean value whether the *i*th action is selected (*True*) or not (*False*).

```

function setActionsRequiredConstraints(actionsRequired List<List<Integer>>)
    (actions[actionsRequired[i][j]] = true AND actions[actionsRequired[i][j+1]] = true AND ...)
    OR (actions[actionsRequired[i+1][j]] = true AND actions[actionsRequired[i+1][j+1]] = true AND ...)
    OR (actions[actionsRequired[i+2][j]] = true AND actions[actionsRequired[i+2][j+1]] = true AND ...)
    ...

```

Fig. 9. Building of the constraints related to the required actions.

- *costs* is an array where each position *i* represents either the cost of applying the action indicated by *actions[i]*, or is set to 0 if no action is needed.
- *TDV* is an array in which each position *i* represents the row ID of the target value BR.DV according to the action indicated by *actions[i]*.
- *TDQM* is an array where each position (*tdqm₁*, *tdqm₂*, ...) represents the row triggered after the evaluation of a DMN table related to a specific dimension (e.g., Completeness, Consistency.).
- *TDQA* is an Integer variable that represents the row ID of the output of the BR.DQA evaluation.
- *TDUD* is an Integer variable that represents the row ID of the output of the BR.DUD evaluation.

4.3. Objective function and constraints

The objective function minimises the summation of the array *costs* of every action applied. Lastly, following Fig. 7, the **Constraints** are of two types:

- *C_{dmn}* represents a set of constraints built according to the DMN model. The process of generating constraints from a DMN model has previously been studied in the literature (Feldman, 2016), and reference is made to these techniques in building our COP. These constraints are intended to bind the domain of the variables *TDQM* and *TDQA*, according to the values of *TDV* using propagation. It also sets the value of *TDUD* according to the selected *targetDud*.
- *C_{action}* involves constraints that describe whether each action *action[i]* is selected (true) or not (false). If an action is selected, then it sets the cost of such action in *cost[i]*. The consequences and requirements are as follows. **Consequences.** The list of consequences of that action (*IActions[i]*) is looped. Therefore, for each consequence, the output value of the affected BR.DV is set according to the attribute *rowTriggered* of the consequence (i.e., *TDV[consequence.brdv] = consequence.rowTriggered*).

4.4. Actions required

These are built from the attribute *actionsRequired* of the action being processed. To better explain how these constraints are built, a function (*setActionsRequiredConstraints*) has been created that takes the list of lists of Integers *actionsRequired*. Fig. 9 summarises how the constraints are built. In short, *actionsRequired[i]* is the *i*th list, and *actionsRequired[i][j]* is the *j*th element of the *i*th list, which is an Integer value indicating the corrective action required. Each *i*th list indicates a possible combination of actions that must be selected altogether.

5. Validating DMN4DQ+ through a real case study

This section presents a case study designed to evaluate the feasibility, scalability, and practical value of the DMN4DQ+. Since the proposal builds upon the existing DMN4DQ methodology, we reuse the case study analysed in our previous work (Valencia-Parra et al., 2021). This existing study aimed to (i) demonstrate the feasibility and scalability of the approach through a scalable set of tools implementing the methodology, and (ii) validate its applicability in a real-world use case. In this section, that validation is extended to DMN4DQ+, providing a specific implementation of the proposed extension. We demonstrate its feasibility and

analyse the impact and added value of DMN4DQ+ in the same use case. We highlight how the extension enhances decision-making support and cost-benefit reasoning capabilities compared to the original framework.

The scenario is based on Amazon Web Services cloud server instance dataset. Due to data quality issues, some instances in the catalogue cannot be sold, since listed server instances may not correspond to real server instances. Validation involves a large-scale application over 1,048,570 records to demonstrate the practical feasibility and traceability of the proposed framework. The dataset, described in Valencia-Parra et al. (2021), contains attributes such as CPU, memory, storage, OS, location, and price for each instance.

All the material employed in this validation, including the code of DMN4DQ+, the implementation of the COP, the DMN model used to evaluate the usability of the data, and the results obtained, are attached to this paper.²

The DMN model used herein refines the model from the referenced study by detailing 13 BR.DV tables with 45 decision rules. It incorporates three data quality dimensions (Completeness, Accuracy, Consistency) and 11 decision rules across 13 inputs. Data quality assessment involves five rules and three inputs, while the BR.DUD hierarchy includes two rules on data record usability, thereby totalling 63 decision rules.

5.1. Feasibility and scalability

To assess the feasibility of our proposal on this use case, a data usability evaluation was conducted and the Elapsed Real-Time (ERT) for COP resolution was measured. The implementation leverages *dmn4spark*³ with COPs solved via Choco-solver.⁴

The metrics that are evaluated are: the different usability profiles found according to the number of records, the ERT consumed per record required to evaluate each usability profile, and the ERT for solving the COPs. These three metrics can help us to evaluate the average number of usability profiles that can be found in a dataset, the consumed time to ascertain the usability profiles, and the consumed time to find the optimal configuration of actions.

First, the usability profile for each of the 1,048,570 dataset records was calculated, yielding 66 profiles based on the DMN model, with 47 marked as *Do not use* and therefore requiring repair. Fig. 10 shows these 66 profiles along the x-axis, where the first 47 are marked as *Do not use*, while the y-axis (logarithmic scale) shows the number of records per profile. Notably, profile 48 contains the most records (519,934), whereas nine profiles (4, 16, 17, 28, 34, 40, 54, 55, and 59) each contain just one record. Overall, 694,361 records are marked as *Use*, while 354,209 records are marked as *Do not use*.

Secondly, Fig. 11 illustrates the efficiency of dataset usability evaluation and usability profile computation, and focuses on average execution time per record per profile. The process involves: (1) evaluating dataset tuples; (2) grouping execution times by usability profile and dividing by record count per profile; and (3) repeating the second step five times to obtain average ERTs. The results indicate no correlation between usability profiles and ERT per record, with an average ERT per record of 0.0003 s. Four profiles (34, 46, 55, and 57) have notably higher ERTs, which can be attributed to their association with very few records (in

² DMN4DQ+ resources: <https://doi.org/10.5281/zenodo.15657538>

³ dmn4spark: <https://github.com/IDEA-Research-Group/dmn4spark>

⁴ Choco-solver: <https://choco-solver.org/>

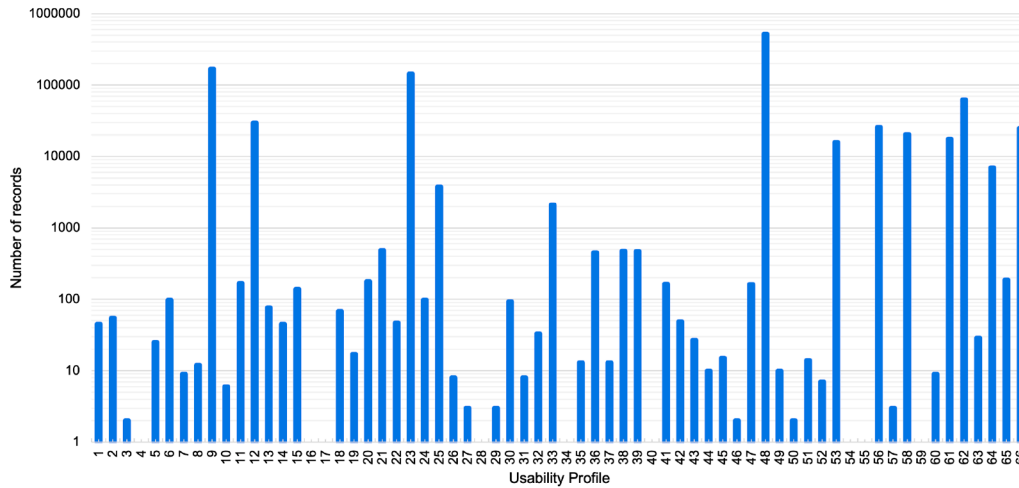


Fig. 10. Number of records within each usability profile after the evaluation.

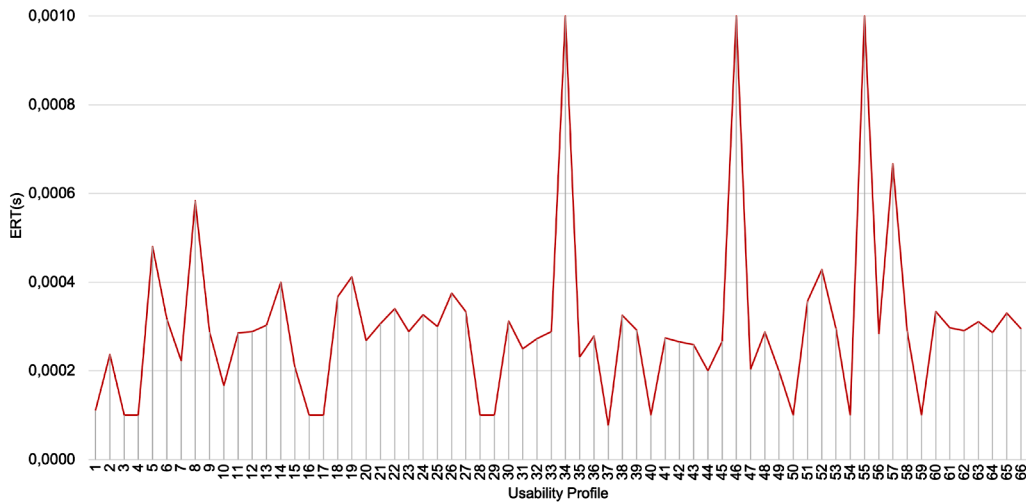


Fig. 11. ERTs per record for the evaluation of each usability profile. The ERTs are the average of five executions.

certain cases, just one record). To evaluate the scalability of the proposal according to the number of records, the potential ERT must be analysed to ascertain the usability profile per tuple. According to the information shown in Fig. 11, this lies between 0.0001 and 0.001 s. Therefore, a dataset of 1,000,000 records can take a maximum of 1000 s, that is approximately 16 min. However, this evaluation can be paralised if the number of records is very high and the total ERT must be reduced.

Thirdly, the feasibility of COP resolution was tested (Fig. 12) with 17 corrective actions. A COP was launched for each usability profile to select the optimal profiles based on corrective action costs, with the experiment repeated five times. Fig. 12 shows the average ERT per profile, with a global average ERT of 39 s. The scalability of the solution is linked to the complexity of solving the Constraint Optimisation Problems. The complexity is related to the number of usability profiles found labelled as ‘Do not use’, 47 in this example, since one COP must be created and solved for each of these profiles. On the other hand, the number of records per usability profile does not impact the evaluation time due to having to find the optimal data repair actions, as Fig. 12 shows.

5.2. Cost-benefit and decision support potential

A total of 47 usability profiles were labelled as “Do not use” according to the business rules defined in the DMN4DQ model. For each of

these profiles, the DMN4DQ+ framework successfully generated a valid solution consisting of one or more corrective actions that can transform the profile into a usable state. This fact results in a reparability rate of 100 % at the profile level. In terms of data coverage, usability profiles that can be successfully repaired imply 354,209 records, which corresponds to approximately 33.7 % of the dataset.

However, although a 33.7 % of the dataset can be repaired, data stakeholders must take into account both the cost and the coverage of each corrective action. Fig. 13 depicts, for each usability profile, the cost for its repair (i.e., changing columns so that all the records with that UP are marked as *usable*), and the number of rows associated with that UP (i.e., the coverage).

The results presented in Table 4 provide a comprehensive summary of the corrective actions suggested by the DMN4DQ+ framework for repair of usability profiles marked as *do not use*. This table illustrates how DMN4DQ+ enables data stakeholders to perform a clear cost-benefit analysis for informed decision making. For instance, data stakeholders can immediately see that applying a single action, Data enrichment, defined in this case study as “contacting the data source to retrieve missing values or enriching it through external joins”, would enable 33 % of the dataset to be repaired. This action alone exerts an impacts on over 346,000 rows. Note that, thanks to the design of DMN4DQ+, the cost of the corrective action that affects these rows is bounded and predictable. The data stakeholder can also understand the impact of actions with minimal cost, such as Data standardisation and Regex string

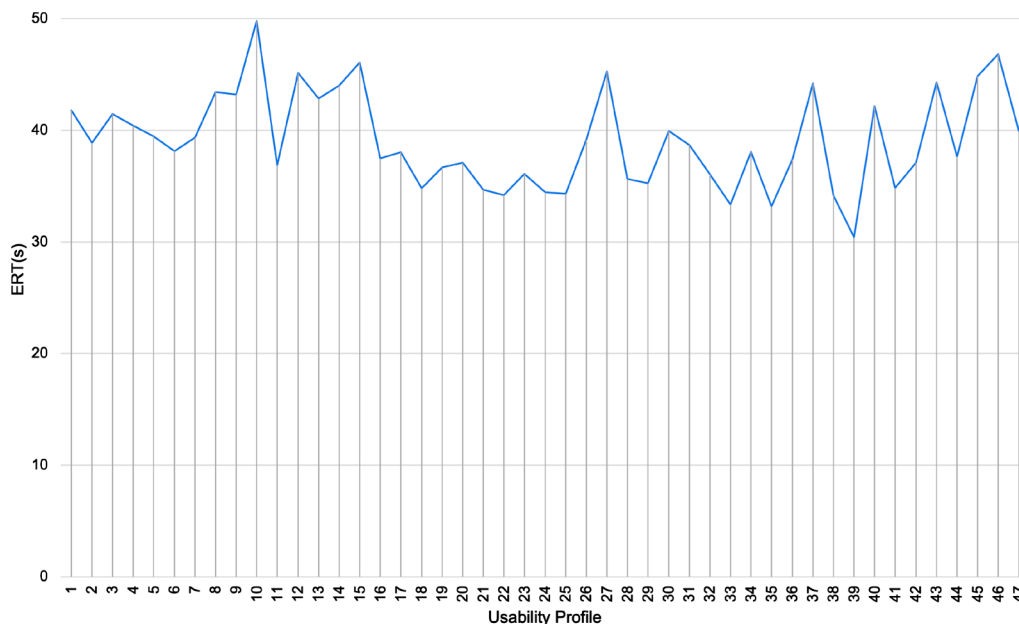


Fig. 12. COP resolution ERTs per usability profile. The ERTs are the average of five executions.

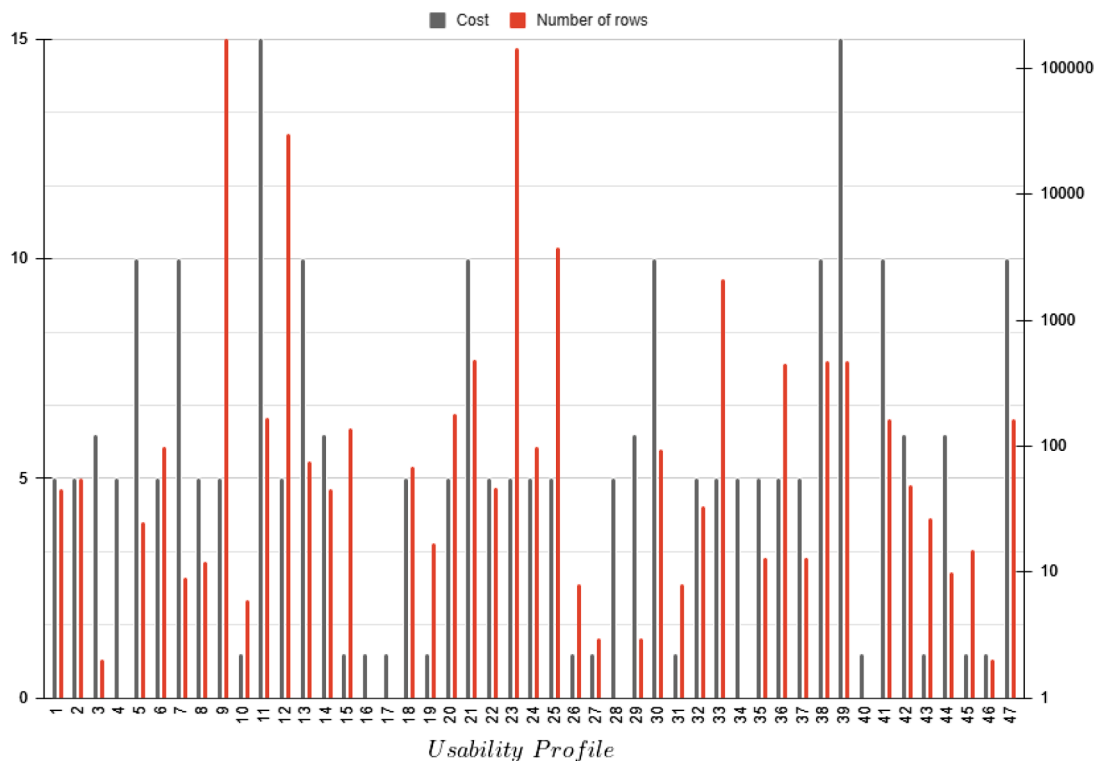


Fig. 13. For each usability profile initially classified as *Do not use*, the figure displays (i) the cost of converting the profile into *usable* (right-hand scale, in grey), and (ii) the number of records associated with that profile in the dataset (left-hand scale, in red).

replacement. In this case study, combining these two actions enables the recovery of 948 rows.

6. Related work

Even when there exist certain specific approaches to data cleansing, such as HoloClean (Boeckling et al., 2022), Renuver (Breve et al., 2022), and Derand (Song et al., 2020) to name just a few, the main focus of this contribution is to provide the adequate means to select the optimal method to enhance of the data. Readers are

therefore encouraged to assume these specific proposals as a means to implement data corrective actions, such as those described in Section 3.1.

In McGilvray (2021), Redman (2016b), the authors recognised that organisations must perform a trade-off analysis between the costs of repairing data quality and the costs derived from the consequences of managing and using data with substandard data quality (Ge & Helfert, 2013). To better carry out this analysis, it could be interesting to analyse both types of costs (Even & Shankaranarayanan, 2009). Unfortunately, very little research has been conducted to

Table 4

Corrective actions required to repair non-usable usability profiles. Each row represents a unique (or combination) of actions identified by the COP as the optimal solution for converting the UPs into *usable*.

Actions	Attributes	Cost	UPs	Rows	% Total
Data enrichment	Clock Speed	5	8	346,625	33.05
Data enrichment	Memory	5	5	2394	0.23
Data enrichment	Operating System	5	7	724	0.07
Data standardisation	Operating System	1	8	222	0.02
Regex string replacement	Memory	1	4	6	0.00
Data enrichment + Regex string replacement	Clock Speed, Memory	6	1	2	0.00
Data enrichment + Data enrichment	Clock Speed, Memory, Operating System	10	8	1492	0.14
Data enrichment + Data enrichment + Data enrichment	Clock Speed, Memory, Operating System	15	2	637	0.06
Data enrichment + Data standardisation	Clock Speed, Operating System	6	4	107	0.01

model these costs. These analyses have typically been conducted *ad hoc*.

It is estimated that the impact of managing and using data with inadequate levels of quality incurs a sizable cost for organisations. It is often difficult to estimate the monetary benefits of improving data quality; several authors have characterised these benefits in terms of increased usability of data by considering that the more usable the data is, the fewer costs are expected to be incurred to fix data with inadequate quality levels (Laney, 2017; Watts et al., 2009). In this respect, several authors have proposed assessing data quality from the perspective of potential usability. Our investigation concludes that there is a pressing need to reduce the cost of improving data quality (e.g., data cleansing) for the maximisation of its usability. This conclusion is drawn by calculating the optimum set of corrective actions to attain sufficient levels of data quality for the required level of usability. However, DMN4DQ+ does not prescribe a specific cost estimation model; consequently, it can benefit from using any of the suitable cost models described in the following paragraphs.

In Ge and Helfert (2013), Ge and Helfert classify the cost of improving data quality according to four main criteria: (1) data quality costs in terms of origin and life cycle; (2) data quality costs per effect; (3) data quality costs per dimension; and (4) data quality costs in terms of progression. To characterise these costs, several authors have proposed different cost estimation models, such as Fan in Fan (2015), who presents a function cost (D, D_r) to represent the efforts required to repair an instance D of a relational schema R . Fan reported the existence of several repair models based on how the function cost is defined: S-repair, C-repair, CC-repair, and U-repair. Each cost estimation model introduces several parameters to model the various aspects involved. For instance, Wijzen (2005) introduces weight-related attributes of the tuple to be repaired and a distance function to model the effort required to fix the value of inadequate quality. On the other hand, Ilyas and Chu (2019), in their contribution towards formalising a cost model, establish the description length DL as a unit for the model M for the length of the model $L(M)$ plus the length to encode the data values in the relation to be improved $L(I|M)$: $DL = L(M) + L(I|M)$. The goal is to find M such that DL is minimised.

Finally, it should be borne in mind that although specific authors introduce methods to find a trade-off between the cost of repairing data quality and minimisation of the impact of problems due to inadequate levels of quality (Laney, 2017), most of these studies are conducted *ad hoc*, and they remain highly focused on specific scenarios of the use of data, with and/or for particular technologies. For instance, Even et al. (2010) explores models for cost-effective data quality management for ERP, and Ardagna et al. (2018) introduces a relation based on Confidence-Cost-Time for Big Data. In this respect, DMN4DQ+ would pioneer a generic method based on optimisation techniques to find the trade-off between the cost of achieving adequate levels of quality and the benefits of using usable data, in accordance with the organisation's appetite for risk. This framework can be used regardless of the context (based on the agnostic definition of business rules) and irrespec-

tive of the technology (it supposes the data to be independent of the technology).

7. Threats to validity

In the following paragraphs, the threats to the validity of the DMN4DQ+ framework are analysed based on the aspects identified in Runeson et al. (2012).

Construct validity: Many of the limitations and threats to the validity of DMN4DQ+ are inherited from those specified for DMN4DQ in Valencia-Parra et al. (2021). Due to the very nature of the DMN and FEEL language, which enables the definition of business rules for individual records in a single table, the main limitation of DMN4DQ and DMN4DQ+ is that they must both be applied record by record from the same table. Hence, there is no support for data quality management involving a set of records within the same table or across different tables. Due to this limitation, our specification states that the only data quality problems that DMN4DQ+ can handle are SAST and MAST problems (see Table 2). However, by establishing the necessary contextualisations, one way to alleviate this situation is by using the concept of a “cluster of records with the same usability profile”. Another concern that could be considered a limitation of the framework is that, by definition, DMN4DQ+ is agnostic to the specific “data quality cost model”. However, this can be a benefit when it comes to generalising the results. Lastly, the selection of the COP as a model to determine the most suitable combination of corrective actions to improve the data usability can be considered a threat to the construct validity. However, we propose that COP is assumed herein as a sufficiently well-proven optimisation framework that applies to most potential scenarios in the scope of DMN4DQ+.

Internal validity: One threat to the internal validity of DMN4DQ+ could be the quality and stability of the business rules used during the generation of recommendations on the usability of the datasets. However, the process of validating the business rules inherited from DMN4DQ alleviates this situation. Another threat to internal validity could involve the capability of the data quality analyst and business expert to define the problem in a way that the COP can find a suitable solution.

External validity: This refers to the potential interest of other practitioners and researchers in the approach. To alleviate this threat, several components have been provided (e.g., the workflow and the technological stack), which could attract their interest. Ideally, their feedback could be obtained to improve DMN4DQ+. Moreover, our proposal does not provide a method to define exhaustively every single business rule for data quality that can exist in an organisation. This means that no mechanisms is offered to evaluate whether the described data quality rules adequately address the entire data quality problem. The list of corrective actions is not exhaustive and may be expanded as more use cases emerge. However, this is not a negative aspect for our proposal; it is expected that feedback from other practitioners and researchers to achieve this aim. The same holds true regarding a potential catalogue of data quality improvement costs.

8. Conclusions and future work

Data quality is a critical issue in the application of decision-making and data analysis. The usability of the data depends on the context of use, the business needs, and the organisation's appetite for risk. To ensure suitable levels of usability, specific corrections may be necessary (e.g., corrective actions) for data whose quality requires improvement. Data corrections can be expensive and may incur wide-reaching ramifications. This paper presents DMN4DQ+ as an extension of DMN4DQ, where the optimal combination of corrective actions can be derived from the application of constraint optimisation techniques based on the data quality rules described in DMN, the cost model of the actions, and the usability profile of the record to be improved. The proposal, conveniently supported by the development of a technological stack, has been evaluated using a real dataset, which shows its applicability and performance. For future work, we propose extending the solution to consider actions that affect multiple records and usability profiles. Moreover, it would be it is interesting to analyse the profiles of the records before assessing the data quality to facilitate the creation of Business Rules for Data Values. We also plan to expand the types of business rules that can be assessed and are involved in a data quality improvement process. Furthermore, Constraint optimisation problems can be solved using an exhaustive analysis, where the applicability of a local search can be a feasible option; for example, the application of genetic algorithms applied to constraint problems. We plan to study this research line in the future.

CRedit authorship contribution statement

Álvaro Valencia-Parra: Conceptualization, Methodology, Software, Validation, Formal analysis, Investigation, Writing – original draft, Visualization; **Ángel Jesús Varela-Vaca:** Conceptualization, Methodology, Supervision, Writing – review & editing; **Luisa Parody:** Formal analysis, Writing – review & editing; **Ismael Caballero:** Resources, Supervision, Writing – review & editing; **María Teresa Gómez-López:** Funding acquisition, Supervision, Writing – review & editing, Project administration.

Data availability

Data will be made available on request.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests: Maria Teresa Gomez-Lopez reports financial support was provided by Spain Ministry of Science and Innovation. Ismael Caballero reports financial support was provided by Spain Ministry of Science and Innovation. Alvaro Valencia-Parra reports financial support was provided by University of Seville. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This research was partially funded by projects AETHER-US (PID2020-112540RB-C44), KOSMOS-US (PID2024-155363OB-C42), ALBA-US (TED2021-130355B-C32) and AETHER-UCLM (PID2020-112540RB-C42), both supported by the Spanish Ministry of Science and Innovation (MCIN/AEI/10.13039/501100011033) and the “European Union NextGenerationEU/PRTR”. It also received funding from Project ADAGIO (SBPLY/21/180501/000061) by the JCCM and FEDER, as well as VI PPIT-US from Universidad de Sevilla.

Appendix. Acronyms

The acronyms are shown in Table A.1.

Table A.1
Acronyms.

Acronym	Meaning
BR.DD	Business Rules for Data Decision
BR.DQA	Business Rules for Data Quality Assessment
BR.DQM	Business Rules for Data Quality Measurement
BR.DUD	Business Rules for Data Usability Decision
BR.DV	Business Rules for Data Values
DQ	Data Quality
COP	Constraint Optimisation Problem
CP	Constraint Programming
CSP	Constraint Satisfaction Problem
DMN	Decision Model and Notation
DMN4DQ	DMN for Data Quality
ERT	Elapsed Real-Time
FEEL	Friendly Enough Expression Language
MAMT	Multiple Attributes in Multiple Tuples
MAST	Multiple Attributes in Single Tuples
SAMT	Single Attributes in Multiple Tuples
SAST	Single Attributes in Single Tuples

References

- Ardagna, D., Cappiello, C., Samá, W., & Vitali, M. (2018). Context-aware data quality assessment for big data. *Future Generation Computer Systems*, 89, 548–562. <https://doi.org/10.1016/j.future.2018.07.014>
- Bertossi, L. (2011). Database repairs and consistent query answering. Morgan & Claypool Publishers.
- Boeckling, T., De Tré, G., & Bronselaer, A. (2022). Cleaning data with selection rules. *IEEE Access*, 10, 125212–125229. <https://doi.org/10.1109/ACCESS.2022.3222786>
- Breve, B., Caruccio, L., Deufemia, V., & Polese, G. (2022). RENUVER: A missing value imputation algorithm based on relaxed functional dependencies. In *Proceedings of the 25th international conference on extending database technology, EDBT 2022, Edinburgh, UK, March 29 - April 1, 2022* (pp. 1:52–1:64). OpenProceedings.org.
- Caballero, I., Gualo, F., Rodríguez, M., & Piattini, M. (2022). BR4DQ: A methodology for grouping business rules for data quality evaluation. *Information Systems*, 109, 102058. <https://doi.org/10.1016/j.is.2022.102058>
- de Almeida, W. G., de Sousa, R. T., de Deus, F. E., Daniel Amvame Nze, G., & de Mendonça, F. L. L. (2013). Taxonomy of data quality problems in multidimensional data warehouse models. In *2013 8th Iberian conference on information systems and technologies (CISTI)* (pp. 1–7). IEEE.
- Even, A., & Shankaranarayanan, G. (2009). Utility cost perspectives in data quality management. *Journal of Computer Information Systems*, 50(2), 127–135. <https://www.tandfonline.com/doi/abs/10.1080/08874417.2009.11645391>
- Even, A., Shankaranarayanan, G., & Berger, P. D. (2010). Evaluating a model for cost-effective data quality management in a real-world CRM setting. *Decision Support Systems*, 50(1), 152–163. <https://doi.org/10.1016/j.dss.2010.07.011>
- Fan, W. (2015). Data quality: From theory to practice. *ACM SIGMOD Record*, 44(3), 7–18. <https://doi.org/10.1145/2854006.2854008>
- Feldman, J. B. (2016). What-if analyzer for DMN-based decision models. *RuleML*.
- Ge, M., & Helfert, M. (2013). Cost and value management for data quality. In S. W. Sadiq (Ed.), *Handbook of data quality, research and practice* (pp. 75–92). Springer. https://doi.org/10.1007/978-3-642-36257-6_4
- Geibinger, T., Mischek, F., & Musliu, N. (2024). Investigating constraint programming and hybrid methods for real world industrial test laboratory scheduling. *Journal of Scheduling*, 27(6), 607–622. <https://doi.org/10.1007/S10951-024-00821-0>
- Green, A. F., Beck, J. C., & Coles, A. J. (2024). Using constraint programming for disjunctive scheduling in temporal AI planning. In P. Shaw (Ed.), *30th international conference on principles and practice of constraint programming, CP 2024, September 2–6, 2024, Girona, Spain* (pp. 12:1–12:17). Schloss Dagstuhl - Leibniz-Zentrum für Informatik (vol. 307). LIPIcs. <https://doi.org/10.4230/LIPICS.CP.2024.12>
- Gualo, F., Rodríguez, M., Verdugo, J., Caballero, I., & Piattini, M. (2021). Data quality certification using ISO/IEC 25012: Industrial experiences. *Journal of Systems and Software*, 176, 110938.
- Ilyas, I. F., & Chu, X. (2019). Data Cleaning (vol. 28). ACM Books. ACM. <https://doi.org/10.1145/3310205>
- ISO 8000-61 (2022). Data quality — Part 61: Data quality management: Process reference model. Standard International Organization for Standardization.
- Jiménez-Piqueras, C., Ruiz, R., Parreño-Torres, C., & Alvarez-Valdés, R. (2023). A constraint programming approach for the premarshalling problem. *European Journal of Operational Research*, 306(2), 668–678. <https://doi.org/10.1016/J.EJOR.2022.07.042>
- Kellerer, H., Pferschy, U., & Pisinger, D. (2004). Knapsack problems. Springer Berlin Heidelberg. <https://doi.org/10.1007/978-3-540-24777-7>
- Laghriissi, A., Retal, S., & Idrissi, A. (2016). Modeling and optimization of the network functions placement using constraint programming. In D. E. Boubiche, H. Hamdan, & A. Bounceur (Eds.), *Proceedings of the international conference on big data and*

- advanced wireless technologies, BDAW 2016, Blagoevgrad, Bulgaria, November 10-11, 2016 (pp. 52:1-52:8). ACM. <https://doi.org/10.1145/3010089.3010137>
- Laney, D. B. (2017). Infonomics: How to monetize, manage, and measure information as an asset for competitive advantage. Routledge.
- McGilvray, D. (2021). Executing data quality projects: Ten steps to quality data and trusted information (TM). Academic Press.
- Oliveira, P., Rodrigues, F., & Henriques, P. R. (2005). A formal definition of data quality problems. In F. Naumann, M. Gertz, & S. E. Madnick (Eds.), *Proceedings of the 2005 international conference on information quality (MIT ICIQ conference), November 10-12, 2006*. MIT.
- OMG. (2019). Decision model and notation (DMN). Object Management Group Version 1.2. <https://www.omg.org/spec/DMN>
- Pérez-Álvarez, J. M., Maté, A., Gómez-López, M. T., & Trujillo, J. (2018). Tactical business-process-decision support based on KPIs monitoring and validation. *Computers in Industry*, 102, 23-39. <https://doi.org/10.1016/j.compind.2018.08.001>
- Quindroit, P., Fruchart, M., Degoul, S., Perichon, R., Martignè, N., Soula, J., Marcilly, R., & Lamer, A. (2023). Definition of a practical taxonomy for referencing data quality problems in health care databases. *Methods of Information in Medicine*, 62(01/02), 019-030.
- Redman, T. (2016a). Bad data costs the U.S. \$3 trillion per year. *Harvard Business Review*. <https://hbr.org/2016/09/bad-data-costs-the-u-s-3-trillion-per-year>
- Redman, T. C. (2016b). Getting in front on data: Who does what. Technics Publications.
- Rossi, F., van Beek, P., & Walsh, T. (Eds.) (2006a). Handbook of constraint programming (vol. 2). Foundations of artificial intelligence. Elsevier. <https://www.sciencedirect.com/science/bookseries/15746526/2>
- Rossi, F., van Beek, P., & Walsh, T. (2006b). Handbook of constraint programming. Elsevier.
- Runeson, P., Höst, M., Rainer, A., & Regnell, B. (2012). Case study research in software engineering - guidelines and examples. Wiley.
- Shankaranarayanan, G., & Cai, Y. (2006). Supporting data quality management in decision-making. *Decision Support Systems*, 42(1), 302-317.
- Song, S., Sun, Y., Zhang, A., Chen, L., & Wang, J. (2020). Enriching data imputation under similarity rule constraints. *IEEE Transactions on Knowledge and Data Engineering*, 32(2), 275-287. <https://doi.org/10.1109/TKDE.2018.2883103>
- Subramanian, V. (2025). Data quality engineering. In *Applied machine learning for data science practitioners*, pp. 135-172). Wiley Data and Cybersecurity. <https://doi.org/10.1002/9781394155408.ch6>
- Urli, T., & Kilby, P. (2017). Constraint-based fleet design optimisation for multi-compartment split-delivery rich vehicle routing. In J. C. Beck (Ed.), *Principles and practice of constraint programming - 23rd international conference, CP 2017, Melbourne, Vic, Australia, August 28 - September 1, 2017, proceedings* (pp. 414-430). Springer (vol. 10416). Lecture Notes in Computer Science. https://doi.org/10.1007/978-3-319-66158-2_27
- Valencia-Parra, Á., Parody, L., Varela-Vaca, Á. J., Caballero, I., & Gómez-López, M. T. (2019). DMN for data quality measurement and assessment. In *BPM 2019 International workshops, Vienna, Austria, September 1-6, 2019* (pp. 362-374). Springer (vol. 362). LNBP.
- Valencia-Parra, Á., Varela-Vaca, Á. J., Gómez-López, M. T., & Ceravolo, P. (2019). CHAMELEON: Framework to improve data wrangling with complex data. In *40th International conference on information systems, ICIS 2019*. Association for Information Systems. https://aisel.aisnet.org/icis2019/data_science/data_science/16
- Valencia-Parra, Á., Parody, L., Varela-Vaca, Á. J., Caballero, I., & Gómez-López, M. T. (2021). DMN4DQ: When data quality meets DMN. *Decision Support Systems*, 141, 113450.
- Varela-Vaca, Á. J., Rosado, D. G., Sánchez, L. E., Gómez-López, M. T., Gasca, R. M., & Fernández-Medina, E. (2021). CARMEN: A framework for the verification and diagnosis of the specification of security requirements in cyber-physical systems. *Computers in Industry*, 132, 103524. <https://doi.org/10.1016/J.COMPIND.2021.103524>
- Wang, J., Liu, Y., Li, P., Lin, Z., Sindakis, S., & Aggarwal, S. (2024). Overview of data quality: Examining the dimensions, antecedents, and impacts of data quality. *Journal of the Knowledge Economy*, 15(1), 1159-1178.
- Watts, S., Shankaranarayanan, G., & Even, A. (2009). Data quality assessment in context: A cognitive perspective. *Decision Support Systems*, 48(1), 202-211.
- Wijzen, J. (2005). Database repairing using updates. *ACM Transactions on Database Systems*, 30(3), 722-768. <https://doi.org/10.1145/1093382.1093385>
- Woodall, P., Oberhofer, M. A., & Borek, A. (2014). A classification of data quality assessment and improvement methods. *International Journal of Information Quality*, 3(4), 298-321.